Predicting Pressure–Volume–Temperature (PVT) and Thermodynamic Properties of Real Gases Using the Peng–Robinson Equation of State (EOS) and Machine Learning (`realgas-ml` v0.3.0)

Satwik Gupta (Roll No. `2025CH11012`) & **Udhav Jaipuria** (Roll No. `2025CH11729`)

Department of Chemical Engineering · Chemical Engineering Thermodynamics Project · November 2026 · GitHub Repository: `realgas_ml`

---

1. Introduction and Thermodynamic Equations

In this project for our Chemical Engineering Thermodynamics course, we built `realgas-ml` (v0.3.0) (`pyproject.toml`, `README.md`), a self-contained Python package and web application that predicts the Pressure–Volume–Temperature (*PVT*), molar enthalpy (h), molar entropy (s), and fugacity (φ, f) properties of 15 pure real gases in the supercritical region. Instead of calling external property libraries like CoolProp or Cantera at runtime, our code generates *65,536* state points per gas (*983,040* points in total) directly from the Peng and Robinson (1976) cubic Equation of State (PR76 EOS) combined with National Aeronautics and Space Administration (NASA) 7-coefficient ideal-gas heat-capacity polynomials. For each gas, we train a four-output local Radial Basis Function (`LocalRBFRegressor`) Machine Learning (ML) model over a normalized temperature and log-pressure grid so that properties can be predicted quickly and accurately without solving a cubic equation every time.

In our analytical solver `realgas_ml/peng_robinson.py`, pressure P (*Pa*) is related to absolute temperature T (*K*) and molar volume v ($m^3\ mol^{-1}$) by the Peng–Robinson (1976) Equation of State (EOS):

$$P(T,v) = \frac{RT}{v-b} - \frac{a(T)}{v^2+2bv-b^2}, \quad a(T) = a_c\left[1+\kappa\left(1-\sqrt{\frac{T}{T_c}}\right)\right]^2, \quad b = 0.07780\,\frac{RT_c}{P_c} \tag{1}$$

where $R = 8.31446261815324\ J\ mol^{-1}\ K^{-1}$ is the universal gas constant, $a_c = 0.45724\ R^2T_c^2/P_c$ is the attraction parameter at the critical point, T_c and P_c are the critical temperature and critical pressure, and $\kappa = 0.37464 + 1.54226\ \omega - 0.26992\ \omega^2$ depends on Pitzer's acentric factor ω. Differentiating $a(T)$ with respect to temperature gives the derivative needed for enthalpy and entropy departure calculations, $(da)/(dT) = -(a_c\ \kappa\ g(T))/(T_c\sqrt{(T/T_c)})$, where $g(T) = 1 + \kappa(1 - \sqrt{(T/T_c)})$. Using the dimensionless parameters $A = (a(T)P)/(R^2T^2)$ and $B = (bP)/(RT)$, Equation (1) is written as the standard cubic polynomial in terms of the compressibility factor $Z = (Pv)/(RT)$:

$$Z^3 + (B-1)Z^2 + (A - 3B^2 - 2B)Z + (-AB + B^2 + B^3) = 0 \tag{2}$$

The function `cubic_z(A, B)` in `peng_robinson.py` solves Equation (2) at each (T,P) point using `numpy.roots`, keeps only physically valid real roots with $|Im(r)| < 10^{-9}$ and $Re(r) > B$ (which ensures $v > b$), and picks the largest real root corresponding to the single-phase gas root.
To calculate real-gas enthalpy, entropy, and fugacity departures (Maginn, CBE 20260), we define the logarithmic term L:

$$L = \ln\left[\frac{Z+(1+\sqrt{2})B}{Z+(1-\sqrt{2})B}\right] = \operatorname{log1p}\left[\frac{2\sqrt{2}\,B}{Z+(1-\sqrt{2})B}\right] \tag{3}$$

Computing L with `np.log1p` in `peng_robinson.py` avoids subtracting two nearly equal numbers at low pressures ($P \to 1\ bar$, where $B \ll 1$ and $Z \to 1$), which prevents floating-point rounding errors. Using L, the molar enthalpy departure h_{dep}, molar entropy departure s_{dep}, log fugacity coefficient $\ln \varphi$, and fugacity f follow standard textbook departure formulas:

$$h_{\mathrm{dep}}(T,P) = RT(Z-1) + \frac{T(da/dT) - a(T)}{2\sqrt{2}\,b}\,L, \qquad s_{\mathrm{dep}}(T,P) = R\ln(Z-B) + \frac{da/dT}{2\sqrt{2}\,b}\,L \tag{4}$$

$$\ln\phi(T,P) = Z - 1 - \ln(Z-B) - \frac{A}{2\sqrt{2}\,B}\,L, \qquad f(T,P) = \phi(T,P)\,P \tag{5}$$

The ideal-gas changes $h_{ig}(T)$ and $s_{ig}(T)$ are calculated in `ideal_integrals(T, info)` by analytically integrating the first five coefficients $(a_0,\ldots,a_4)$ of the NASA 7-coefficient polynomial $C_p°(T)/R = \sum_{k=0}^{4} a_k\,T^k$ from the reference state $T_{ref} = 300.0\ K$ and $P_{ref} = 10^5\ Pa$ (*1 bar*), where we set $h_{ig}(T_{ref}) = 0$ and $s_{ig}(T_{ref}, P_{ref}) = 0$:

$$\tilde{h}(T) = R\left(a_0T + \tfrac{a_1}{2}T^2 + \tfrac{a_2}{3}T^3 + \tfrac{a_3}{4}T^4 + \tfrac{a_4}{5}T^5\right), \quad \tilde{s}(T) = R\left(a_0\ln T + a_1T + \tfrac{a_2}{2}T^2 + \tfrac{a_3}{3}T^3 + \tfrac{a_4}{4}T^4\right) \tag{6}$$

We leave out the standard formation constants (a_5, a_6) and integrate across the low- and high-temperature NASA polynomial intervals (*[200, 1000] K* and *[1000, 6000] K*, or *[300, 1000, 5000] K* for SO_2) using `np.clip` so there is no jump at *1000 K* (which matters for H_2O since its temperature range goes up to *1100.06 K*). Adding the departure terms gives the total molar enthalpy $h(T,P) = h_{ig}(T) + h_{dep}(T,P)$ and total molar entropy $s(T,P) = s_{ig}(T) - R\ln(P/P_{ref}) + s_{dep}(T,P)$. Because the real-gas departure $h_{dep}(300\ K, 1\ bar)$ is not zero, total h and s are slightly non-zero at *300 K* and *1 bar*.

2. Selected Gases and Operating Ranges

The fluid database in `realgas_ml/data/metadata.json` includes 15 common pure gases: noble gases (*He*, *Ne*, *Ar*), diatomic gases (H_2, N_2, O_2, *CO*), triatomic and polar gases (CO_2, SO_2, H_2O, NH_3), and straight-chain alkanes from methane to *n*-butane (CH_4, C_2H_6, C_3H_8, $n\text{-}C_4H_{10}$). The NASA 7-coefficient heat-capacity parameters are taken from Cantera 3.2.0 `nasa_gas.yaml` (`SHA-256: 4de6199d...8667e4db`, based on NASA Technical Memorandum 4513). Critical constants (T_c, P_c, ω, M) are taken from Cantera 3.2.0 `critical-properties.yaml` (citing Lemmon et al., 2022, National Institute of Standards and Technology (NIST) Chemistry WebBook Standard Reference Database (SRD) 69; Yaws, 2001; and Poling et al.) for nine gases, and from CoolProp fluid reference tables for Helium, Argon, Neon, Ammonia, SulfurDioxide, and Water.

Table 1 summarizes the temperature and pressure bounds we chose for each gas. Every fluid covers the same pressure range of $P \in [10^5, 5\times10^6]\ Pa$ (*1–50 bar*), while the minimum temperature is set to $T_{min} = max(300.0\ K, 1.10\ T_c)$. Keeping $T \geq 1.10\ T_c$ ensures that every state point stays in the single-phase supercritical region well above the critical temperature, avoiding two-phase vapor–liquid splitting and the infinite compressibility slope $(\partial v/\partial P)_T \to -\infty$ at the critical point. Even so, for heavier hydrocarbons like *n*-Butane ($P_c = 37.60\ bar$), Propane ($P_c = 42.50\ bar$), and Ethane ($P_c = 48.80\ bar$), a pressure of *50 bar* corresponds to reduced pressures $P_r = P_{max}/P_c$ of *1.33*, *1.18*, and *1.02* at $T_r = 1.10$. In this region near the critical point, intermolecular attraction causes strong real-gas deviations ($Z \approx 0.65$–0.85) and steeper changes in h_{dep} and $\ln\varphi$. On the other hand, light gases like *He*, H_2, and *Ne* are at very high reduced temperatures ($T_r \in [6.76, 115.49]$), where molecular repulsion dominates ($Z > 1$) and the property surfaces are almost flat.

Table 1: Critical constants (molar mass M, critical temperature T_c, critical pressure P_c, Pitzer acentric factor ω), operating temperature bounds (T_{min}, T_{max} at pressure $P \in [1.0, 50.0]$ bar), reduced temperature range ($T_r = T/T_c$), and data sources for all 15 pure gases in `metadata.json`.

Fluid Name	Formula	M (g/mol)	T_c (K)	P_c (bar)	ω (—)	T_{min} (K)	T_{max} (K)	$T_r = T/T_c$ Range	Critical / NASA Data Source
Helium	He	4.0026	5.1953	2.2832	*-0.38354*	300.00	600.00	*57.74–115.49*	CoolProp He / L10/90
Hydrogen	H_2	2.0159	32.9800	12.9300	*-0.21700*	300.00	600.00	*9.10–18.19*	Cantera (Poling) / TPIS78
Neon	Ne	20.1790	44.4000	26.6163	*-0.03549*	300.00	600.00	*6.76–13.51*	CoolProp Ne / L10/90
Nitrogen	N_2	28.0135	126.2000	33.9000	*0.03800*	300.00	600.00	*2.38–4.75*	Cantera (NIST) / TPIS78
CarbonMonoxide	CO	28.0101	133.4000	35.0000	*0.06600*	300.00	499.00	*2.25–3.74*	Cantera (NIST) / TPIS79
Argon	Ar	39.9480	150.6870	48.6300	*-0.00219*	300.00	600.00	*1.99–3.98*	CoolProp Ar / L 6/88
Oxygen	O_2	31.9988	154.5800	50.4300	*0.02200*	300.00	600.00	*1.94–3.88*	Cantera (NIST) / TPIS89
Methane	CH_4	16.0428	190.7000	46.3000	*0.01100*	300.00	600.00	*1.57–3.15*	Cantera (NIST) / L 8/88
CarbonDioxide	CO_2	44.0098	304.2000	73.9000	*0.22800*	334.62	600.00	*1.10–1.97*	Cantera (Yaws) / L 7/88
Ethane	C_2H_6	30.0690	305.4000	48.8000	*0.09900*	335.94	600.00	*1.10–1.96*	Cantera (NIST) / L 8/88
Propane	C_3H_8	44.0956	369.7000	42.5000	*0.15300*	406.88	628.81	*1.10–1.70*	Cantera (NIST) / L 6/90
Ammonia	NH_3	17.0305	405.5600	113.6339	*0.25569*	446.12	689.45	*1.10–1.70*	CoolProp NH_3 / TPIS89
n-Butane	$n\text{-}C_4H_{10}$	58.1222	425.6000	37.6000	*0.19900*	468.16	574.00	*1.10–1.35*	Cantera (NIST) / L 6/90
SulfurDioxide	SO_2	64.0638	430.6400	78.8658	*0.25613*	473.70	524.00	*1.10–1.22*	CoolProp SO_2 / J 6/61
Water	H_2O	18.0153	647.0960	220.6400	*0.34429*	711.81	1100.06	*1.10–1.70*	CoolProp H_2O / L 8/89

3. Machine Learning Model and Software Implementation

Our Machine Learning (ML) pipeline in `train.py`, `realgas_ml/regressor.py`, and `realgas_ml/model.py` is designed to handle two practical issues: pressure varies by a factor of 50 across the domain, and both compressibility factor Z and fugacity coefficient φ must always be positive. Instead of feeding raw (T, P) values into the model, `train.py` and `model.py` scale any state $(T, P) \in [T_{min}, T_{max}] \times [P_{min}, P_{max}]$ onto the unit square $x = (x_1, x_2) \in [0, 1]^2$ using linear temperature scaling and logarithmic pressure scaling:

$$x_1(T) = \frac{T - T_{\min}}{T_{\max} - T_{\min}} \in [0,1], \qquad x_2(P) = \frac{\ln(P/P_{\min})}{\ln(P_{\max}/P_{\min})} \in [0,1] \tag{7}$$

Scaling pressure logarithmically helps in two ways. First, because the ideal-gas entropy contains the term $-R \ln(P/P_{ref})$, using $\ln P$ as the input coordinate turns that logarithmic curve into a straight line in x_2, which the polynomial part of our model fits exactly. Second, spacing grid points evenly in $\ln P$ puts more training points at lower pressures (*1–10 bar*) instead of crowding them all near *50 bar*. The model is trained on the four-output target vector $y = [\ln Z, h, s, \ln \varphi]^\top \in \mathbb{R}^4$. During prediction in `GasModel.predict`, we take the exponential of the predicted $\ln Z$ and $\ln \varphi$ values ($Z = \exp(\hat{y}_1)$ and $\varphi = \exp(\hat{y}_4)$) so that $Z > 0$ and $\varphi > 0$ are guaranteed, and then compute all other molar and total properties for n moles:

$$v = \frac{ZRT}{P}, \quad V = nv, \quad \rho = \frac{M}{v}, \quad H = n\hat{y}_2, \quad S = n\hat{y}_3, \quad f = \phi P \tag{8}$$

As checked in `tests/test_properties.py` and `tests/test_app.py`, fugacity f is an intensive property and does not change when the number of moles n changes, whereas total volume V, total enthalpy H, and total entropy S scale linearly with n.
In `realgas_ml/regressor.py`, `LocalRBFRegressor` wraps SciPy's `RBFInterpolator` inside a standard scikit-learn estimator class. First, the four target columns are standardized to zero mean and unit variance with `StandardScaler`. For any query point $x \in [0, 1]^2$, the interpolator finds the $k = 48$ closest training points $x_{i1}, \ldots, x_{ik}$ and builds a local quintic Radial Basis Function (RBF) interpolation with a degree-2 quadratic polynomial trend and zero smoothing ($\lambda = 0$):

$$\tilde{\mathbf{y}}(\mathbf{x}) = \sum_{j=1}^{48} \mathbf{w}_j \, \varphi\left(\|\mathbf{x} - \mathbf{x}_{i_j}\|_2\right) + \sum_{\alpha_1 + \alpha_2 \le 2} \mathbf{c}_{\alpha_1, \alpha_2} \, x_1^{\alpha_1} x_2^{\alpha_2}, \quad \varphi(r) = -r^5 \tag{9}$$

where the weights $w_j \in \mathbb{R}^4$ and polynomial coefficients $c_{\alpha1,\alpha2} \in \mathbb{R}^4$ pass through all *48* local training points ($\tilde{y}(x_{ij}) = y_{std}(x_{ij})$) along with *6* polynomial side conditions $\sum_{j=1}^{48} w_j \, x_{ij,1}^{\alpha_1} \, x_{ij,2}^{\alpha_2} = 0$ for $\alpha_1 + \alpha_2 \le 2$. Trying to fit a global Radial Basis Function (RBF) model across all $N = 65{,}536$ points at once would require solving a $65{,}536 \times 65{,}536$ dense matrix (*~ 34 GB* of memory), which is not practical on a normal computer. Using only the $k=48$ nearest neighbors reduces each prediction to a fast k-dimensional (k-d) tree lookup and a small 54×54 linear system. Each saved model file (`realgas_ml/data/.joblib`, compressed with level 3) takes only *2.63–2.66 MB* on disk.
When temperature T and molar volume v are given instead of pressure, `GasModel.pressure(T, v)` in `model.py` solves $\hat{v}(T, P) - v = 0$ for P using Brent's root-finding method (`scipy.optimize.brentq`) over $[10^5, 5 \times 10^6]$ *Pa*. Because molar volume always decreases with pressure at constant temperature ($(\partial v/\partial P)_T < 0$) in the single-phase gas region, Brent's method reliably finds the pressure to within a relative error of 10^{-8} (tested in `test_properties.py`). We provided four ways to use the package: the Python Application Programming Interface (API) (`example.py`), the Command-Line Interface (CLI) tool `realgas-ml` (`realgas_ml/cli.py`), a script to export NumPy Zip (NPZ) datasets to Comma-Separated Values (CSV) files (`export_data.py`), and an interactive Streamlit web application (`app.py`, configured with `.streamlit/config.toml` and `run_app.bat`) that supports unit conversions (*K*/°C, *bar*/*kPa*/*MPa*/*Pa*), *(T,P)* and *(T,v)* modes, pressure plots, and CSV downloads.

4. Model Testing, Accuracy, and Thermodynamic Checks

In `train.py`, we used a fixed random seed (`np.random.default_rng(20261016 + index)`) to create four separate datasets for each gas: (i) the $256 \times 256 = 65{,}536$ training grid on $[0, 1]^2$; (ii) *512* random interior validation points to track learning curves at grid sizes of $N \in 32^2, 64^2, 128^2, 256^2$; (iii) *2,048* random interior test points; and (iv) *256* boundary test points (*64* random points along each of the four edges $x_1 \in 0,1$ and $x_2 \in 0,1$). For strictly positive properties (molar volume v and fugacity f), we report the **Mean Absolute Percentage Error (MAPE)** and **Maximum Absolute Percentage Error (Max APE)**, calculated from the log predictions as $100\,|\exp(\hat{y} - y) - 1|$. For molar enthalpy h and molar entropy s (which pass through zero near *300 K* and *1 bar*), we report the **Mean Absolute Error (MAE)** and **Maximum Absolute Error (Max AE)**.

Table 2: Interior Test Split (N = 2,048) and Boundary Test Split (N = 256) errors comparing our Machine Learning (ML) models against direct Peng–Robinson calculations for all 15 gases trained on the 256 × 256 (65,536-point) grid (from `realgas_ml/data/*_metrics.json`). Abbreviations: **MAPE** = Mean Absolute Percentage Error, **Max APE** = Maximum Absolute Percentage Error, **MAE** = Mean Absolute Error, **Max AE** = Maximum Absolute Error.

Fluid (Metrics File)	Int v MAPE (%)	Int v Max APE (%)	Int f MAPE (%)	Int f Max APE (%)	Int h MAE (J/mol)	Int s MAE (J/mol·K)	Bnd v MAPE (%)	Bnd v Max APE (%)	Bnd f MAPE (%)	Bnd f Max APE (%)	Bnd h Max AE (J/mol)	Bnd s Max AE (J/mol·K)
Helium	*3.46e-10*	*4.94e-08*	*2.80e-10*	*3.90e-08*	*1.02e-08*	*2.55e-10*	*1.23e-09*	*1.38e-07*	*9.38e-10*	*9.68e-08*	*2.19e-06*	*1.22e-08*
Neon	*5.49e-10*	*7.84e-08*	*5.66e-10*	*8.28e-08*	*1.98e-08*	*3.14e-10*	*5.25e-10*	*2.59e-08*	*5.46e-10*	*2.56e-08*	*7.37e-07*	*1.73e-08*
Hydrogen	*6.47e-10*	*7.74e-08*	*6.10e-10*	*7.26e-08*	*3.23e-08*	*3.32e-10*	*8.94e-10*	*6.60e-08*	*7.94e-10*	*4.95e-08*	*2.44e-06*	*2.82e-09*
Water	*9.96e-10*	*8.60e-08*	*9.53e-10*	*7.32e-08*	*4.53e-07*	*4.37e-10*	*1.47e-09*	*3.34e-08*	*1.14e-09*	*2.36e-08*	*6.54e-06*	*6.53e-09*
Nitrogen	*1.16e-09*	*3.12e-07*	*1.06e-09*	*2.46e-07*	*4.16e-08*	*4.22e-10*	*1.84e-09*	*5.90e-08*	*1.83e-09*	*5.43e-08*	*2.09e-06*	*2.23e-08*
Oxygen	*1.30e-09*	*1.90e-07*	*1.10e-09*	*1.20e-07*	*6.17e-08*	*4.57e-10*	*2.20e-09*	*9.14e-08*	*2.09e-09*	*7.17e-08*	*1.77e-06*	*1.78e-08*
CarbonMonoxide	*1.37e-09*	*2.81e-07*	*1.11e-09*	*1.72e-07*	*4.30e-08*	*1.78e-10*	*2.07e-09*	*1.98e-07*	*1.54e-09*	*1.05e-07*	*1.95e-05*	*6.15e-08*
Argon	*1.39e-09*	*2.72e-07*	*1.19e-09*	*1.78e-07*	*6.19e-08*	*3.25e-10*	*1.87e-09*	*3.31e-08*	*1.81e-09*	*3.18e-08*	*8.65e-07*	*5.36e-09*
Methane	*1.85e-09*	*1.60e-07*	*1.45e-09*	*8.28e-08*	*1.16e-07*	*5.04e-10*	*1.06e-08*	*1.37e-06*	*7.71e-09*	*8.10e-07*	*3.61e-05*	*2.00e-07*
Ammonia	*2.27e-09*	*2.14e-07*	*2.09e-09*	*1.87e-07*	*4.53e-07*	*6.38e-10*	*5.90e-09*	*5.11e-07*	*3.24e-09*	*2.50e-07*	*4.70e-05*	*9.10e-08*
CarbonDioxide	*4.78e-09*	*9.96e-07*	*2.71e-09*	*4.80e-07*	*3.78e-07*	*7.16e-10*	*2.47e-08*	*2.84e-06*	*9.46e-09*	*7.60e-07*	*8.98e-05*	*3.59e-07*
SulfurDioxide	*1.07e-08*	*1.98e-06*	*5.74e-09*	*7.47e-07*	*1.43e-06*	*2.40e-09*	*1.09e-08*	*4.29e-07*	*5.34e-09*	*1.62e-07*	*4.74e-05*	*8.64e-08*
Ethane	*1.73e-08*	*7.53e-06*	*5.38e-09*	*1.06e-06*	*8.64e-07*	*2.17e-09*	*5.28e-08*	*2.25e-06*	*1.39e-08*	*4.26e-07*	*1.75e-04*	*4.74e-07*
Propane	*1.85e-08*	*5.63e-06*	*4.65e-09*	*4.77e-07*	*1.09e-06*	*2.23e-09*	*4.66e-08*	*8.53e-07*	*1.09e-08*	*1.53e-07*	*6.82e-05*	*1.62e-07*
n-Butane	*4.08e-08*	*9.48e-06*	*6.99e-09*	*3.41e-07*	*2.16e-06*	*3.89e-09*	*8.28e-08*	*1.04e-05*	*1.12e-08*	*1.50e-06*	*4.10e-04*	*8.24e-07*
Global Worst Case	*4.08e-08*	*9.48e-06*	*6.99e-09*	*1.06e-06*	*2.16e-06*	*3.89e-09*	*8.28e-08*	*1.04e-05*	*1.39e-08*	*1.50e-06*	*4.10e-04*	*8.24e-07*

Table 2 and Figure 2 show that across all 15 gases on both the Interior Test Split and Boundary Test Split, the largest percentage error (Maximum Absolute Percentage Error, *Max APE*) in molar volume is $1.0365 \times 10^{-5}\%$ (*n*-Butane boundary), the largest percentage error (*Max APE*) in fugacity is $1.4985 \times 10^{-6}\%$ (*n*-Butane boundary), the largest absolute error (Maximum Absolute Error, *Max AE*) in molar enthalpy is $4.0967 \times 10^{-4}\ J\ mol^{-1}$, and the largest absolute error (*Max AE*) in molar entropy is $8.2417 \times 10^{-7}\ J\ mol^{-1}\ K^{-1}$. All of these worst-case errors are 1,000 to 10,000 times smaller than the pass/fail tolerances checked in `tests/test_properties.py` (*Max APE* $< 0.01\%$ for v and f, *Max AE* $< 0.5\ J\ mol^{-1}$ for h, and *Max AE* $< 0.005\ J\ mol^{-1}\ K^{-1}$ for s).

The differences in error between gases in Table 2 make physical sense. Because our local Radial Basis Function (RBF) model includes a quadratic polynomial ($m=2$), it fits linear and quadratic trends exactly, so the remaining interpolation error depends on how sharply the property surface curves (third- and higher-order derivatives). For light gases like Helium ($T_r \geq 57.7$), Neon ($T_r \geq 6.76$), and Hydrogen ($T_r \geq 9.10$), the compressibility factor $Z(T,P)$ is very close to *1* and almost flat, giving tiny interior volume Mean Absolute Percentage Errors (MAPEs) between $3.46 \times 10^{-10}\%$ and $6.47 \times 10^{-10}\%$. Water also has a very small interior volume Mean Absolute Percentage Error (MAPE of $9.96 \times 10^{-10}\%$) despite being a polar molecule because *50 bar* is much lower than its critical pressure ($P_{max}/P_c = 50 / 220.64 = 0.2266$ at $T \geq 711.81\ K$), so steam behaves fairly smoothly over this range. In contrast, *n*-Butane, Propane, and Ethane reach $P_{max}/P_c \in [1.02, 1.33]$ near $T_{min} = 1.10\ T_c$, where $Z(T,P)$ bends more steeply near the critical region. Even for those heavier hydrocarbons, boundary errors are only *2–5* × larger than interior errors.

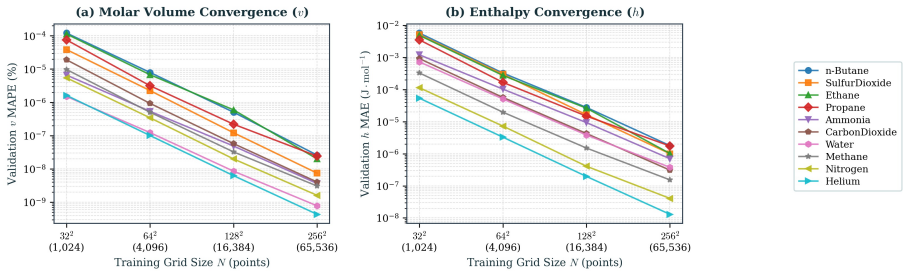


Figure 1: Validation learning curves ($N_{val} = 512$) for grid sizes $N \in \{32^2, 64^2, 128^2, 256^2\}$ showing (a) molar volume Mean Absolute Percentage Error (MAPE) and (b) molar enthalpy Mean Absolute Error (MAE), where doubling the grid resolution per side reduces error by roughly 12× to 18×.

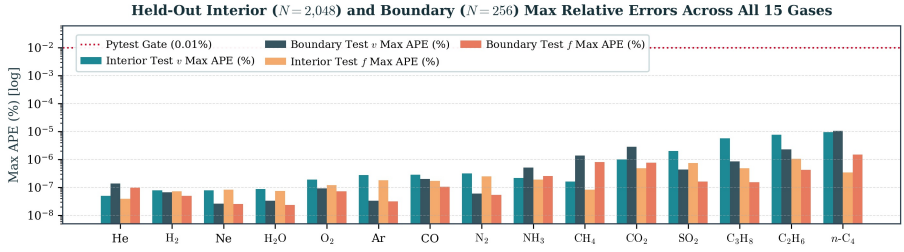


Figure 2: Comparison of Interior Test (N = 2,048) and Boundary Test (N = 256) Maximum Absolute Percentage Error (Max APE) on molar volume v and fugacity f across all 15 gases at N = 65,536, compared against the 0.01% threshold in `test_properties.py`.

Table 3: Validation learning-curve results (N_{val} = 512) showing Molar Volume Mean Absolute Percentage Error (MAPE, %) and Molar Enthalpy Mean Absolute Error (MAE, J/mol) across training grid sizes $N \in \{1{,}024, 4{,}096, 16{,}384, 65{,}536\}$ for all 15 gases, along with the empirical convergence order $p_{eff} = \log_2(\rho_{4\times})$.

Fluid	v MAPE (N=1,024)	v MAPE (N=4,096)	v MAPE (N=16,384)	v MAPE (N=65,536)	p_{eff}(v)	h MAE (N=1,024)	h MAE (N=4,096)	h MAE (N=16,384)	h MAE (N=65,536)	p_{eff}(h)
Helium	*1.62e-06*	*1.01e-07*	*6.28e-09*	*4.33e-10*	*3.95*	*5.38e-05*	*3.28e-06*	*1.96e-07*	*1.29e-08*	*4.01*
Water	*1.50e-06*	*1.23e-07*	*8.55e-09*	*7.80e-10*	*3.64*	*7.28e-04*	*5.10e-05*	*3.83e-06*	*3.77e-07*	*3.64*
Neon	*1.82e-06*	*1.07e-07*	*8.70e-09*	*5.99e-10*	*3.86*	*7.12e-05*	*4.35e-06*	*3.56e-07*	*2.51e-08*	*3.82*
Hydrogen	*2.79e-06*	*1.83e-07*	*1.35e-08*	*1.28e-09*	*3.70*	*1.50e-04*	*9.14e-06*	*6.08e-07*	*4.87e-08*	*3.86*
Oxygen	*3.62e-06*	*2.60e-07*	*2.34e-08*	*1.88e-09*	*3.64*	*1.17e-04*	*6.68e-06*	*5.19e-07*	*5.63e-08*	*3.68*
Argon	*4.85e-06*	*2.65e-07*	*1.75e-08*	*1.73e-09*	*3.82*	*1.18e-04*	*9.52e-06*	*6.12e-07*	*6.33e-08*	*3.62*
CarbonMonoxide	*5.41e-06*	*3.83e-07*	*2.47e-08*	*2.07e-09*	*3.78*	*1.13e-04*	*7.06e-06*	*5.34e-07*	*5.56e-08*	*3.66*
Nitrogen	*5.45e-06*	*3.44e-07*	*2.00e-08*	*1.61e-09*	*3.91*	*1.14e-04*	*7.28e-06*	*4.08e-07*	*4.06e-08*	*3.82*
Ammonia	*6.60e-06*	*5.38e-07*	*4.87e-08*	*3.65e-09*	*3.61*	*1.21e-03*	*1.03e-04*	*9.37e-06*	*6.98e-07*	*3.59*
Methane	*9.70e-06*	*5.04e-07*	*3.24e-08*	*3.07e-09*	*3.87*	*3.29e-04*	*1.97e-05*	*1.51e-06*	*1.53e-07*	*3.69*
CarbonDioxide	*1.90e-05*	*9.29e-07*	*5.76e-08*	*4.04e-09*	*4.06*	*9.23e-04*	*5.69e-05*	*4.27e-06*	*3.08e-07*	*3.85*
SulfurDioxide	*3.85e-05*	*2.26e-06*	*1.22e-07*	*7.47e-09*	*4.11*	*5.14e-03*	*2.99e-04*	*1.64e-05*	*1.00e-06*	*4.11*
Propane	*7.52e-05*	*3.16e-06*	*2.20e-07*	*2.42e-08*	*3.86*	*3.54e-03*	*1.67e-04*	*1.49e-05*	*1.74e-06*	*3.66*
Ethane	*1.13e-04*	*6.76e-06*	*5.94e-07*	*2.00e-08*	*4.16*	*4.88e-03*	*2.75e-04*	*2.63e-05*	*1.02e-06*	*4.08*
n-Butane	*1.21e-04*	*7.91e-06*	*5.06e-07*	*2.60e-08*	*4.06*	*5.76e-03*	*3.27e-04*	*2.77e-05*	*1.79e-06*	*3.88*

Table 3 and Figure 1 show how the validation error drops as we increase the grid size from *32 × 32* (*1,024* points) up to *256 × 256* (*65,536* points). Every time the number of grid points per side is doubled (which cuts the grid spacing Δx in half), the validation Mean Absolute Percentage Error (*MAPE*) and Mean Absolute Error (*MAE*) drop by a factor of *12.2* × to *17.9* × , giving an effective convergence rate of $p_{eff} \in [3.59, 4.16]$. Going from *1,024* points to *65,536* points reduces the prediction errors by more than *2,000* × to *5,600* × .

To make sure our Peng–Robinson equations are thermodynamically consistent, `tests/test_properties.py` runs 47 automated tests across all 15 gases at $T_{mid} = (T_{min} + T_{max})/2$ and $P = 2.0$ *MPa* using central finite differences ($\Delta T = 0.01$ *K*, $\Delta P = 100.0$ *Pa*). These tests verify five standard thermodynamic relations from class to within relative tolerances of 10^{-10} to 2×10^{-5}:

$$g_{\text{dep}} = h_{\text{dep}} - Ts_{\text{dep}} = RT\ln\phi,\ \left(\frac{\partial h}{\partial P}\right)_T = v - T\left(\frac{\partial v}{\partial T}\right)_P,\ \left(\frac{\partial s}{\partial P}\right)_T = -\left(\frac{\partial v}{\partial T}\right)_P,\ \left(\frac{\partial \ln f}{\partial P}\right)_T = \frac{v}{RT},\ \left(\frac{\partial h}{\partial T}\right)_P = T\left(\frac{\partial s}{\partial T}\right)_P \tag{10}$$

5. Conclusion and Limitations

Our `realgas-ml` v0.3.0 project shows that combining log-pressure scaling and log-target transforms (*ln Z, ln φ*) with local quintic Radial Basis Function (RBF) interpolation (*k=48, m=2*) reproduces analytical Peng–Robinson (1976) Equation of State (PR76 EOS) and NASA 7-coefficient polynomial property calculations across 15 pure gases to 8–10 significant digits (Maximum Absolute Percentage Error, *Max APE* ≤ *1.04 × 10⁻⁵%*). By avoiding repeated cubic root solving during forward lookups and using Brent's method for inverse *P(T,v)* calculations, the package gives fast, self-contained predictions for coursework and process calculations. Still, two important limitations noted in `README.md` and `model.py` should be kept in mind. First, because the four outputs *[ln Z, h, s, ln φ]* are fitted as separate columns, the Machine Learning (ML) model's derivatives satisfy the Maxwell relations in Equation (10) up to the *~ 10⁻⁸* interpolation error rather than exactly; calculations that need exact derivative identities should call `peng_robinson.properties` directly. Second, our low test errors measure how closely the Machine Learning (ML) model matches the Peng–Robinson (1976) Equation of State (PR76 EOS), not experimental lab data; PR76 itself is an approximate cubic Equation of State (EOS) and can have noticeable physical error for strongly polar fluids (*H_2O, NH_3*) or light quantum gases (*He, H_2*).

References and Project Links

1. Peng, D.-Y., & Robinson, D. B. (1976). A New Two-Constant Equation of State. *Industrial & Engineering Chemistry Fundamentals*, 15(1), 59–64. https://doi.org/10.1021/i160057a011
2. McBride, B. J., Gordon, S., & Reno, M. A. (1993). *Coefficients for Calculating Thermodynamic and Transport Properties of Individual Species*. National Aeronautics and Space Administration (NASA) Technical Memorandum 4513. https://ntrs.nasa.gov/citations/19940013151
3. Cantera Developers (2025). *Cantera 3.2.0 Species Thermodynamic Polynomials* (Species Thermo Reference), `nasa_gas.yaml`, and `critical-properties.yaml`.
4. Lemmon, E. W., Bell, I. H., Huber, M. L., & McLinden, M. O. (2022). Thermophysical Properties of Fluid Systems. In *National Institute of Standards and Technology (NIST) Chemistry WebBook, NIST Standard Reference Database (SRD) Number 69*. https://doi.org/10.18434/T4D303
5. Bell, I. H., et al. (CoolProp Fluid Documentation): Static critical parameters for Helium, Neon, Argon, Ammonia, SulfurDioxide, and Water.
6. Maginn, E. J. *CBE 20260 Lecture Notes: Fugacity from Equations of State*. University of Notre Dame. CBE20260 Fugacity-EOS Notes
7. Project Repository (`realgas_ml`): `README.md`, `pyproject.toml`, `peng_robinson.py`, `regressor.py`, `model.py`, `train.py`, `app.py`, `cli.py`, `export_data.py`, `metadata.json`, `test_properties.py`, and `test_app.py`.